

Relatório Técnico - Iteração 03

Sistema de Gestão de Supermercado

Grupo: 1DB_3

Tomás Bastos (1251506), Igor Almeida (1251054),
Guilherme Asevedo (1251002), and Rui Silva (1251443)

ISEP - Licenciatura em Engenharia de Telecomunicações e Informática

Abstract. Este relatório documenta a fase de design e implementação do sistema de gestão de inventário e vendas para um supermercado, desenvolvido em C++17 com programação orientada a objetos. O documento apresenta a arquitetura em camadas (Model-View-Controller estendido com Services, Repository, DTOs e Mappers), diagramas de design das classes implementadas, e os casos de uso concretizados nesta iteração. A aplicação utiliza persistência em ficheiros CSV e interface de linha de comandos.

Palavras-chave: C++, MVC, Design de Software, UML, Gestão de Supermercado, Arquitetura em Camadas, DTO, Singleton.

1 Introdução

O objetivo deste projeto é criar uma aplicação para gerir um supermercado, a aplicação vai ter um ADMIN, CAIXAS e CLIENTES.

O ADMIN é responsável por gerir o catálogo da loja, incluindo criar, editar, listar e remover PRODUTOS. Tem também a habilidade de gerir CATEGORIAS de PRODUTOS, configurar PROMOÇÕES temporárias com percentagens de desconto e intervalos de datas, e repor níveis de stock a PRODUTOS que já existem. Adicionalmente, o ADMIN irá gerir contas de CAIXA (criando-as e removendo-as), gerir a base de dados de CLIENTES (registando, editando, listando e removendo CLIENTES), e consultar estatísticas de desempenho globais e por CAIXA, como a faturação e o volume de VENDAS.

O CAIXA pode iniciar e processar VENDAS, adicionando PRODUTOS e quantidades, aplicando descontos automáticos de PROMOÇÕES ativas e concluindo transações com diferentes métodos de pagamento. Pode também associar um CLIENTE a uma VENDA em curso através do seu NIF, consultar o quantidade de PONTOS atual de um CLIENTE e visualizar o seu próprio histórico de desempenho e total faturado. Após concluir uma VENDA, o sistema irá gerar um RECIBO com os detalhes dos PRODUTOS, totais e método de pagamento utilizado.

Os CLIENTES registados no sistema acumularão PONTOS de fidelização com base nas suas compras (ex: 1 ponto por cada 10€ gastos). Estes PONTOS poderão ser utilizados para descontos em compras futuras. O sistema armazenará

os dados do **CLIENTE**, incluindo o seu **NIF**, nome e quantidade de **PONTOS** de fidelização associado.

Este relatório inclui histórias de utilizador, requisitos funcionais e não funcionais (utilizando o modelo **FURPS+**), o modelo de domínio, especificações e diagramas de casos de uso, e o design de software baseado no padrão **Model-View-Controller (MVC)**. A aplicação será desenvolvida em **C++17** utilizando programação orientada a objetos, com persistência de dados através de ficheiros **CSV** externos.

2 Análise de Requisitos

2.1 User Stories

Administrador (ADMIN)

- **US01:** Como **ADMIN**, eu quero criar novos **PRODUTOS** para poder atualizar o catálogo do supermercado.
- **US02:** Como **ADMIN**, eu quero apagar **PRODUTOS** do sistema para que os **PRODUTOS** que já não existam deixem de estar na base de dados.
- **US03:** Como **ADMIN**, eu quero listar todos os **PRODUTOS** existentes para que possa consultar o inventário.
- **US04:** Como **ADMIN**, eu quero aceder a estatísticas globais dos **CAIXAS** para avaliar o desempenho do supermercado.
- **US05:** Como **ADMIN**, eu quero adicionar e remover **CAIXAS** (adição/remoção) para controlar quem tem acesso ao sistema de **VENDAS**.
- **US06:** Como **ADMIN**, eu quero comprar stock para **PRODUTOS** existentes para garantir que não falta **PRODUTOS**.
- **US07:** Como **ADMIN**, eu quero gerir o registo de **CLIENTES** (criar/editar/apagar) para manter a base de dados de **CLIENTES** atualizada.
- **US08:** Como **ADMIN**, eu quero criar e gerir **PROMOÇÕES** (percentagem, data início, data fim) para incentivar as **VENDAS**.
- **US09:** Como **ADMIN**, eu quero gerir **CATEGORIAS** de produtos para organizar o inventário e aplicar impostos diferentemente para cada **CATEGORIA**.

CAIXA

- **US10:** Como **CAIXA**, eu quero listar **PRODUTOS** e consultar preços para informar os **CLIENTES**.
- **US11:** Como **CAIXA**, eu quero realizar **VENDAS** (registando **PRODUTOS** e quantidades) para processar as compras dos **CLIENTES**.
- **US12:** Como **CAIXA**, eu quero concluir **VENDAS** com diferentes métodos de pagamento e emitir **RECIBOS** para documentar a transação.
- **US13:** Como **CAIXA**, eu quero consultar o meu histórico e informação individual para saber o meu desempenho.
- **US14:** Como **CAIXA**, eu quero associar uma **VENDA** a um **CLIENTE** (via **NIF**) para que este possa acumular **PONTOS**.

- **US15:** Como CAIXA, eu quero consultar o quantidade de PONTOS de um CLIENTE para informar o CLIENTE sobre os PONTOS que ele tem.
- **US16:** Como CAIXA, eu quero que o sistema aplique automaticamente os descontos ativos no momento da VENDA sem intervenção manual.
- **US17:** Como CAIXA, eu quero consultar o detalhe de RECIBOS de vendas passadas para esclarecer dúvidas dos clientes.

2.2 Requisitos Funcionais (RF)

- **RF01:** O sistema deve permitir a criação de um novo PRODUTO (nome, preço_base), criando um id único automaticamente.
- **RF02:** O sistema deve permitir a remoção de um PRODUTO pelo seu id.
- **RF03:** O sistema deve listar todos os PRODUTOS com os seus respetivos detalhes e stock existente.
- **RF04:** O sistema deve permitir o registo e remoção de utilizadores com perfil de CAIXA.
- **RF05:** O sistema deve calcular estatísticas de performance (faturação e volume) por CAIXA e globalmente.
- **RF06:** O sistema deve gerir o ciclo de vida de uma VENDA (iniciar, adicionar itens, cancelar, concluir).
- **RF07:** O sistema deve validar a disponibilidade de stock durante o registo de itens na VENDA.
- **RF08:** O sistema deve registar o método de pagamento e mostrar um RECIBO após a conclusão da venda.
- **RF09:** O sistema deve identificar o utilizador e restringir acesso a funcionalidades baseadas em ser um ADMIN ou um CAIXA.
- **RF10:** O sistema deve garantir a persistência de dados em ficheiros externos.
- **RF11:** O sistema deve permitir a gestão de CLIENTES (NIF, Nome) e garantir que o NIF é único.
- **RF12:** O sistema deve permitir associar um CLIENTE a uma VENDA em curso.
- **RF13:** O sistema deve calcular e atribuir PONTOS ao CLIENTE após uma venda concluída (ex: 1 PONTO por cada 10€).
- **RF14:** O sistema deve aplicar automaticamente o resgate de PONTOS para descontos no valor total da VENDA, caso o CLIENTE possua saldo suficiente.
- **RF15:** O sistema deve permitir a criação de PROMOÇÕES com intervalo de datas (início e fim) e percentagem de desconto.
- **RF16:** O sistema deve validar se uma PROMOÇÃO está ativa (data atual dentro do intervalo) antes de aplicar o desconto.
- **RF17:** O sistema deve permitir associar uma PROMOÇÃO a um PRODUTO ou a uma CATEGORIA de produtos.
- **RF18:** O sistema deve garantir que, no caso de existir mais de uma promoções aplicáveis, apenas a que desconte mais dinheiro para o cliente é aplicada.

2.3 Requisitos Não Funcionais (FURPS+)

Functionality (F)

- **RNF01 (Persistência de Dados):** O sistema deve garantir que o armazenamento de dados será em ficheiros CSV locais.

Usability (U)

- **RNF02 (Estrutura de Menus):** O sistema deve permitir uma navegação por menus na CLI, por seleção numérica (0-9).

Reliability (R)

- **RNF03 (Recuperação de Erro):** O sistema deve verificar os tipos de dados de entrada (ex: caracteres em campos numéricos) e pedir uma nova introdução caso o tipo de dado esteja errado, sem perder o estado da transação ativa.
- **RNF04 (Integridade de Dados):** O sistema deve garantir que todas as transações que forem concluídas devem ser registadas num ficheiro.

Performance (P) O sistema deve responder prontamente às interações do utilizador na CLI.

Supportability (S)

- **RNF06 (Portabilidade/Build):** O projeto deve ser compilável através de CMake (versão 3.10 ou superior) em ambiente Linux (GCC/Clang) e Windows (MSVC).

+ (Restrições de Implementação)

- **RNF07 (Norma da Linguagem):** O código fonte deve ser compatível com a norma C++17 (ISO/IEC 14882:2017).

3 Modelação de Domínio

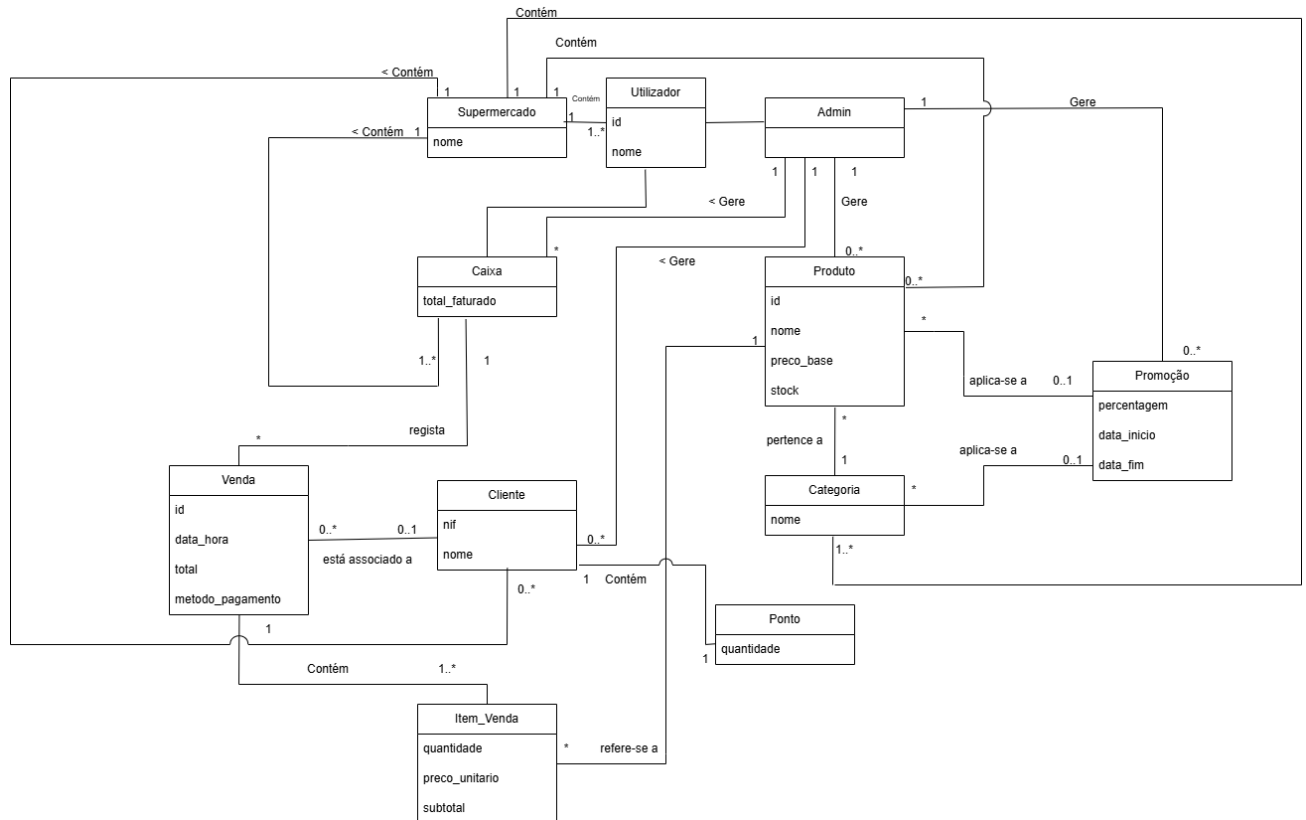


Fig. 1. Diagrama UML de Domínio

4 Design de Software

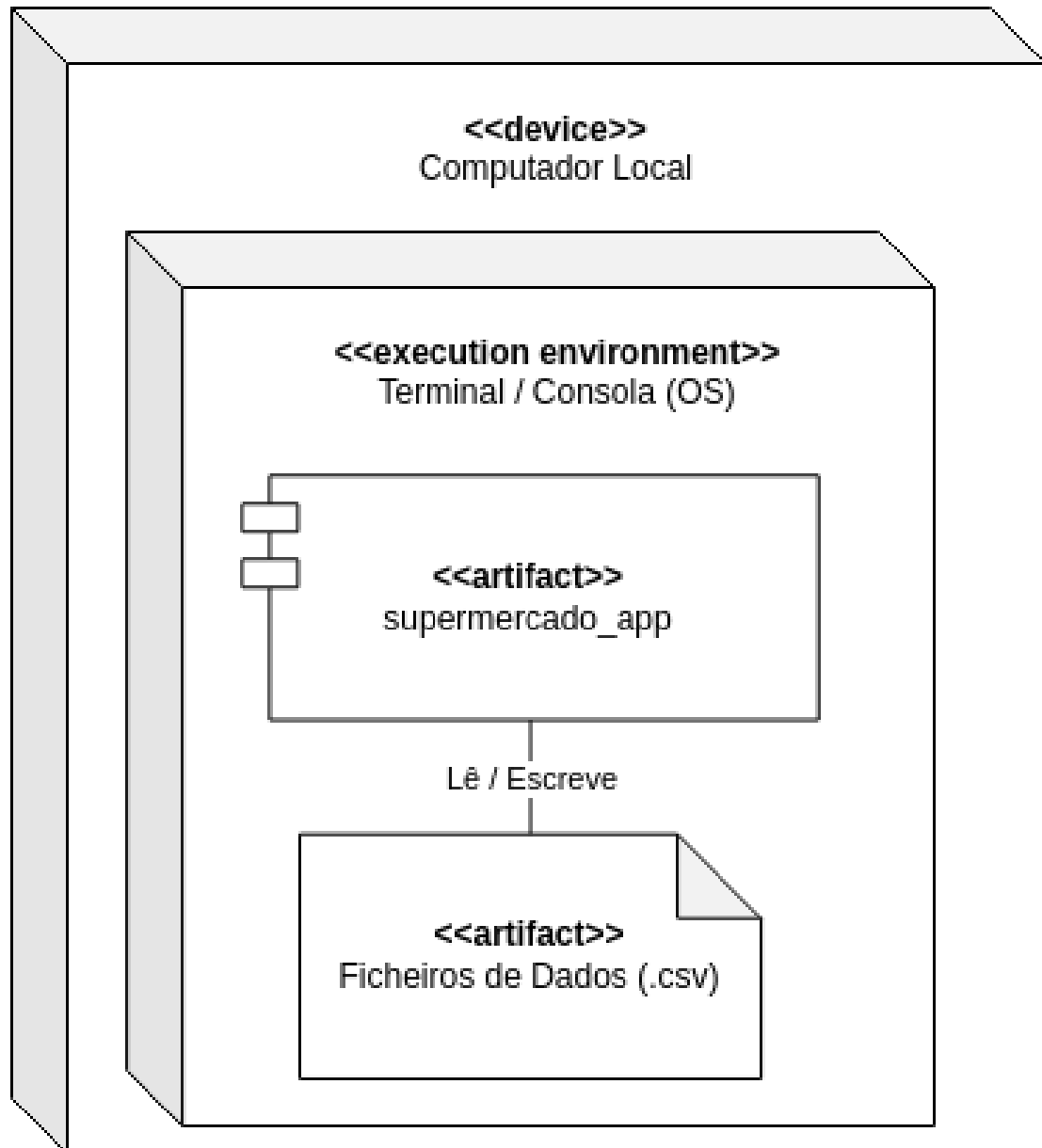


Fig. 2. Diagrama de Arquitetura Física

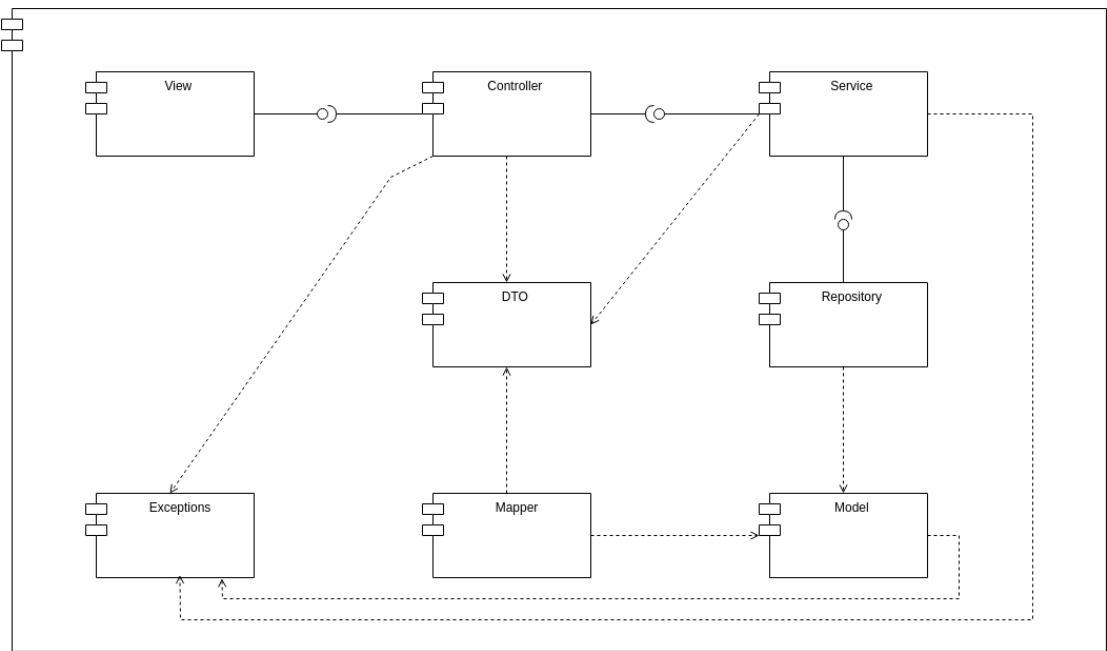


Fig. 3. Diagrama de Arquitetura Lógica

```

Project/
├── main.cpp
├── CMakeLists.txt
├── include/
│   ├── model/
│   │   ├── Admin.h
│   │   ├── Caixa.h
│   │   ├── Categoria.h
│   │   ├── Cliente.h
│   │   ├── ItemVenda.h
│   │   ├── Ponto.h
│   │   ├── Produto.h
│   │   ├── Promocao.h
│   │   ├── Utilizador.h
│   │   └── Venda.h
│   ├── view/
│   │   ├── CaixaView.h
│   │   ├── CatalogoView.h
│   │   ├── CategoriaView.h
│   │   ├── ClienteView.h
│   │   ├── PromocaoView.h
│   │   ├── Utils.h
│   │   ├── VendaView.h
│   │   └── view.h
│   ├── controller/
│   │   └── Controller.h
│   ├── services/
│   │   ├── CaixaService.h
│   │   ├── CategoriaService.h
│   │   ├── ClienteService.h
│   │   ├── ProdutoService.h
│   │   ├── PromocaoService.h
│   │   └── VendaService.h
│   ├── repo/
│   │   └── SupermercadoRepository.h
│   ├── dto/
│   │   ├── CaixaDTO.h
│   │   ├── CategoriaDTO.h
│   │   ├── ClienteDTO.h
│   │   ├── ItemVendaDTO.h
│   │   ├── ProdutoDTO.h
│   │   ├── PromocaoDTO.h
│   │   └── VendaDTO.h
│   ├── mappers/
│   │   ├── CaixaMapper.h
│   │   ├── CategoriaMapper.h
│   │   ├── ClienteMapper.h
│   │   ├── ProdutoMapper.h
│   │   ├── PromocaoMapper.h
│   │   └── VendaMapper.h
│   └── exceptions/
│       ├── DataConsistencyException.h
│       ├── DuplicatedDataException.h
│       ├── InvalidDataException.h
│       └── NoDataException.h
└── src/
    (espelha a estrutura de include/)

```

Fig. 4. Organização do Código

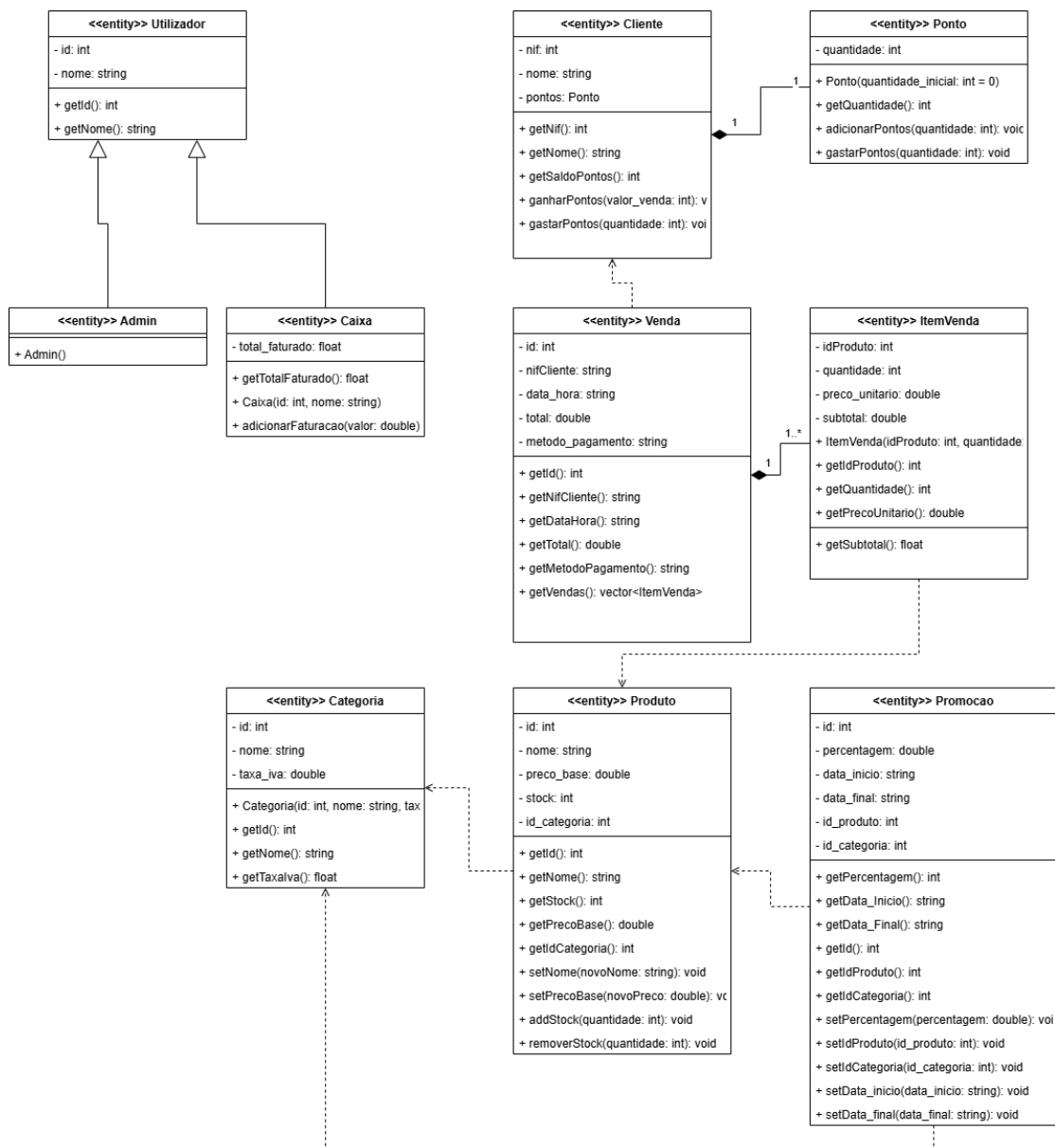


Fig. 5. Diagrama de Classes do Modelo (Design Class Diagram)

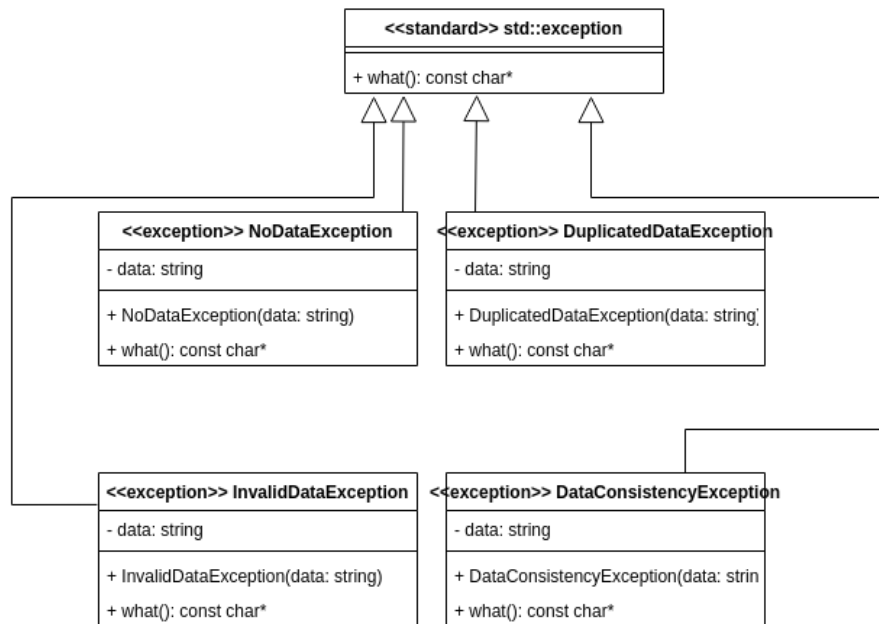


Fig. 6. Diagrama de Classes das Exceções

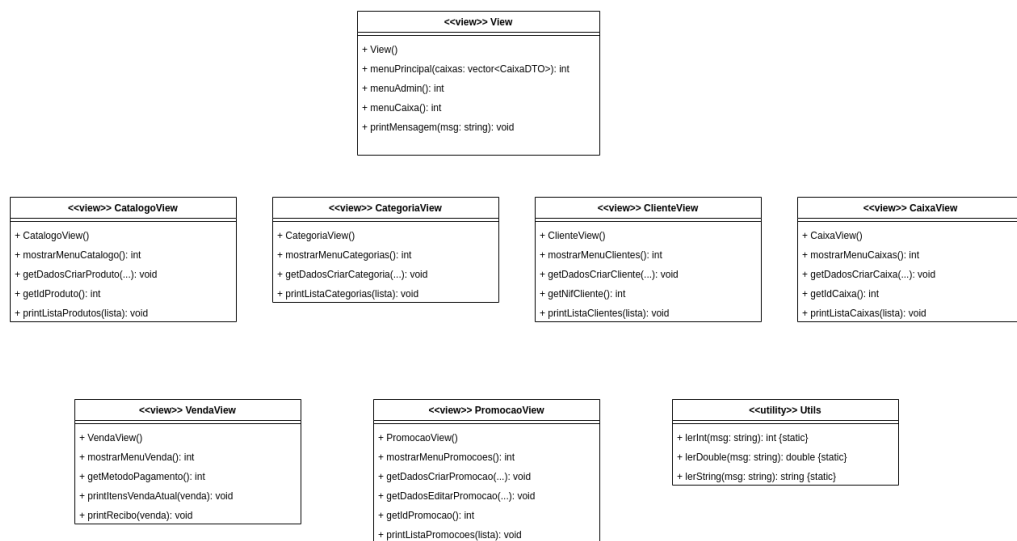


Fig. 7. Diagrama de Classes das Views

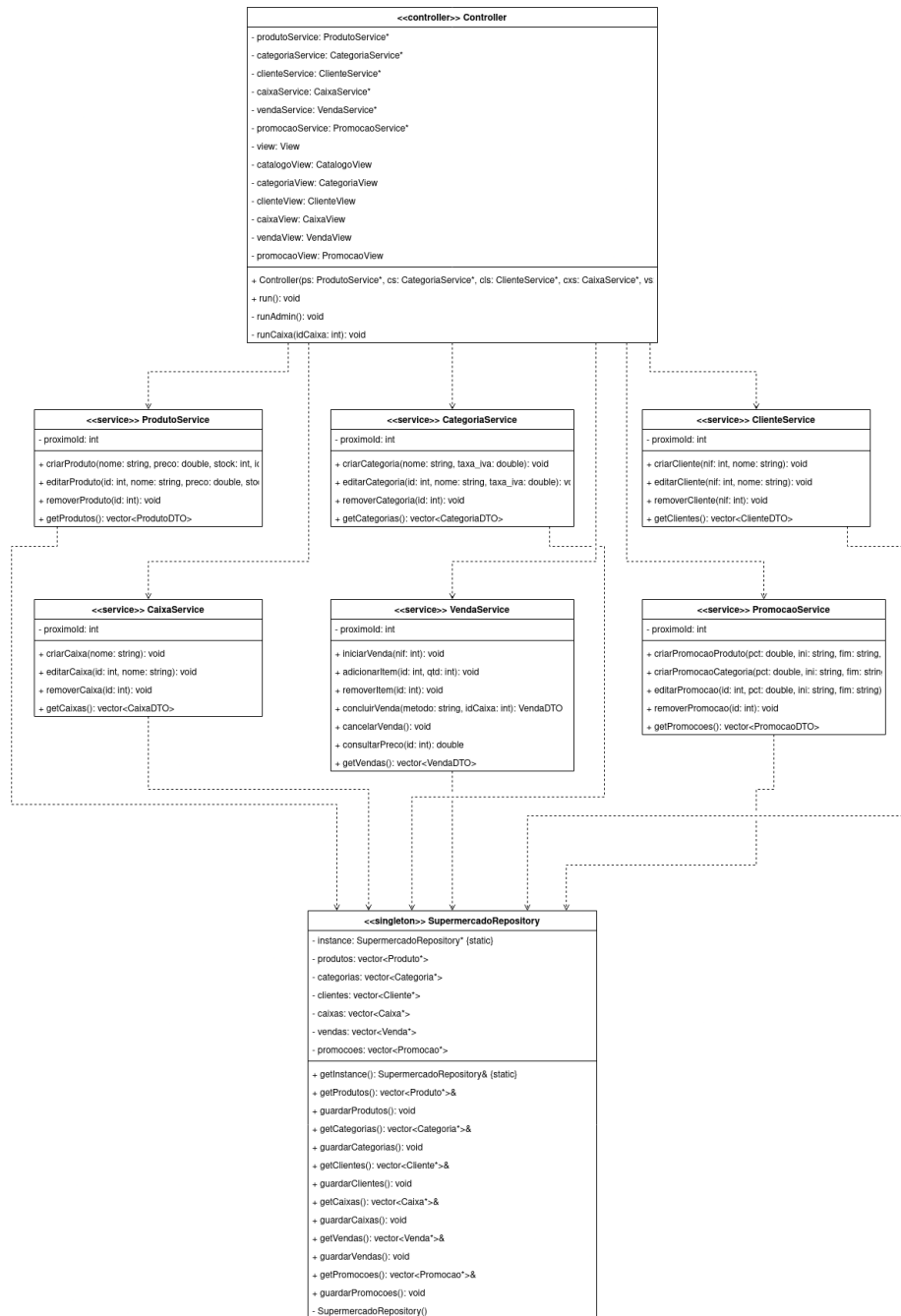


Fig. 8. Diagrama de Classes do Controller, Services e Repository

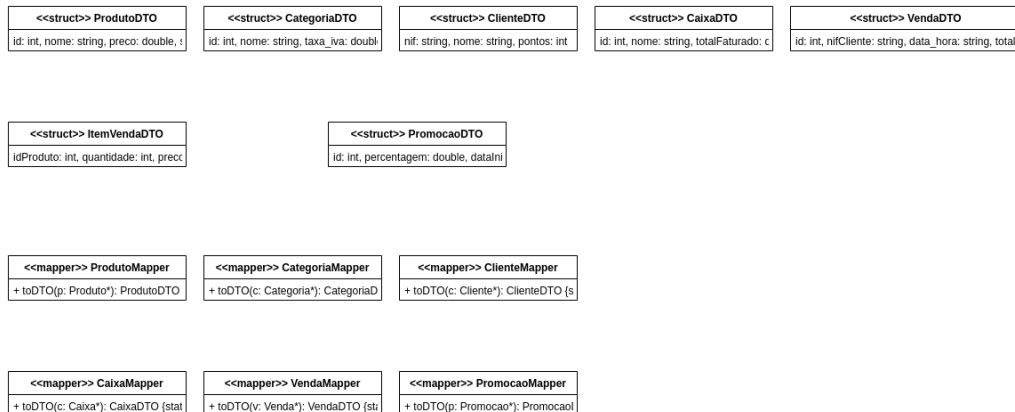


Fig. 9. Diagrama de DTOs e Mappers

4.1 Padrão Model-View-Controller (MVC)

O sistema foi dividido em camadas seguindo o padrão Model-View-Controller que foi estendido com Services, Repository, DTOs e Mappers:

1. **Model:** Contém as classes de dados (Produto, Categoria, Cliente, Venda, ItemVenda, Promocao, Ponto, Caixa, Admin, Utilizador). Estas classes «entity», representam as entidades do domínio e não conhecem a parte do View.
2. **View:** Responsável por toda a interação com o utilizador seja a enviar dados ou a pedir dados. Inclui as classes View, CatalogoView, CategoriaView, ClienteView, CaixaView, VendaView, PromocaoView e Utils (validar o input do utilizador).
3. **Controller:** A classe Controller é a classe que orquestra o funcionamento do sistema. Recebe os Services no construtor, gere a navegação dos menus e deixa todas as operações de negócio aos Services. Trata exceções com blocos try/catch e comunica resultados através da View.
4. **Service:** As classes ProdutoService, CategoriaService, ClienteService, CaixaService, VendaService, PromocaoService contêm a lógica de negócio e validações. Geram IDs automáticos e coordenam a persistência através do Repository.
5. **Repository:** A classe SupermercadoRepository centraliza toda a persistência de dados em ficheiros CSV. É acedida via getInstance() e expõe referências para os vectores internos.
6. **DTO:** Estruturas de transferência de dados: ProdutoDTO, CategoriaDTO, ClienteDTO, CaixaDTO, VendaDTO, ItemVendaDTO, PromocaoDTO que transportam dados entre camadas sem expor os Models diretamente às Views.
7. **Mappers:** Classes ProdutoMapper, CategoriaMapper, ClienteMapper, CaixaMapper, VendaMapper, PromocaoMapper com métodos estáticos toDTO() que convertem Modelos em DTOs.

8. **Exceptions:** Quatro classes de exceção `NoDataException`, `DuplicatedDataException`, `InvalidDataException`, `DataConsistencyException` que herdam de `std::exception` e são usadas pelos `Services` e capturadas pelo `Controller`.

O fluxo de uma operação típica é: **View** recolhe o input, **Controller** chama **Service**, **Service** valida e usa **Repository**, **Mapper** converte Model, DTO, **Controller** passa DTO à **View** para apresentar.

4.2 Arquitetura de Ficheiros

- `include/model/` — Headers das entidades de domínio (`Produto.h`, `Categoria.h`, `Cliente.h`, `Venda.h`, etc.)
- `include/view/` — Headers das classes de interface (`View.h`, `CatalogoView.h`, `CategoriaView.h`, `ClienteView.h`, `CaixaView.h`, `VendaView.h`, `PromocaoView.h`, `Utils.h`)
- `include/controller/` — Header do orquestrador (`Controller.h`)
- `include/services/` — Headers da lógica de negócio (`ProdutoService.h`, `CategoriaService.h`, `ClienteService.h`, `CaixaService.h`, `VendaService.h`, `PromocaoService.h`)
- `include/repo/` — Header do repositório Singleton (`SupermercadoRepository.h`)
- `include/dto/` — Estruturas de transferência de dados (`ProdutoDTO.h`, `CategoriaDTO.h`, etc.)
- `include/mappers/` — Conversores Model-DTO (`ProdutoMapper.h`, `CategoriaMapper.h`, `ClienteMapper.h`, `CaixaMapper.h`, `VendaMapper.h`, `PromocaoMapper.h`)
- `include/exceptions/` — Classes de exceção (`NoDataException.h`, etc.)
- `src/` — Implementações correspondentes a cada header, espelhando a mesma estrutura de pastas.

A compilação é gerida pelo `CMakeLists.txt` (C++17, CMake 3.10+) e o ponto onde o programa começa é o `main.cpp`.

4.3 Padrões de Desenho Aplicados (GRASP)

Exemplos de classes onde usamos certos Padroes de desenho:

- **Creator:** O padrão *Creator* é aplicado na classe `SupermercadoRepository`. No método `carregarProdutos()` ocorre a instanciação dos objetos `Produto` depois da a leitura do ficheiro CSV e extração da informação la dentro, sendo o repositório o agregador que contém e gere as instâncias de produtos.
- **Controller:** O padrão *Controller* é implementado através da classe `Controller`, que atua como o orquestrador do sistema. Esta classe recebe os eventos da interface (View) e dá ordens de execução para as outras camadas.
- **Service:** O padrão *Service* é utilizado na classe `ProdutoService`, onde está a lógica de negócio associada aos produtos (como a atribuição de um ID automático e as verificações de que os dados são válidos).

4.4 Interface de Utilizador

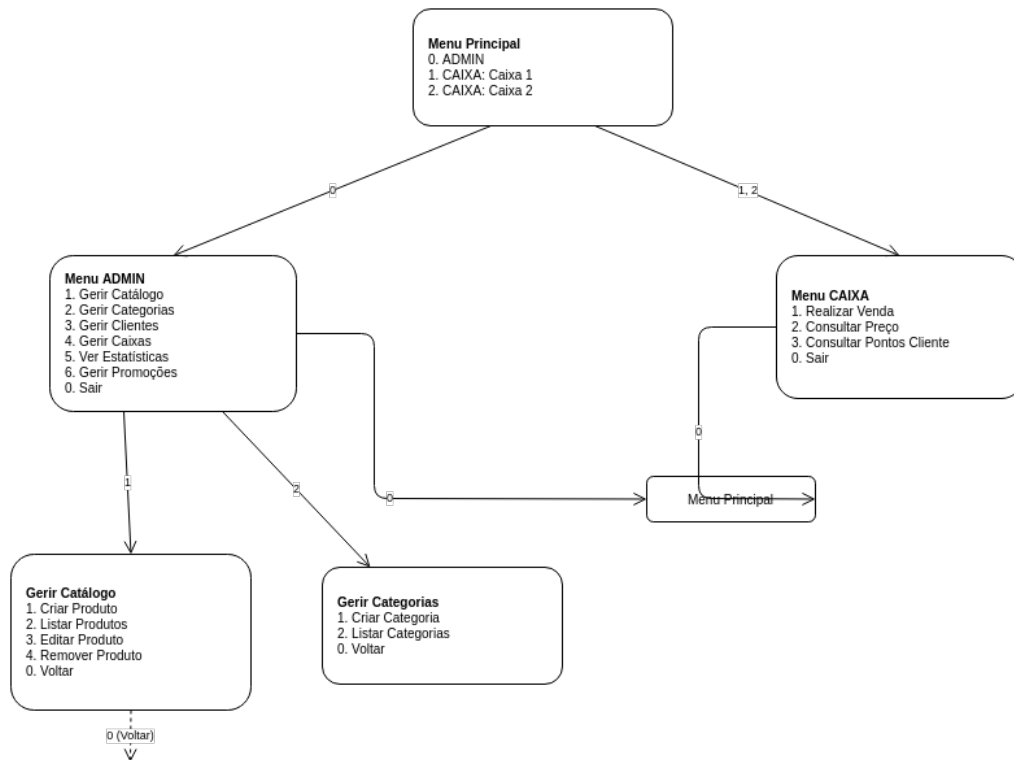


Fig. 10. Árvore de Navegação da Interface de Utilizador

5 Modelo de Casos de Uso

Nesta secção detalhamos as interações entre os atores e o sistema.

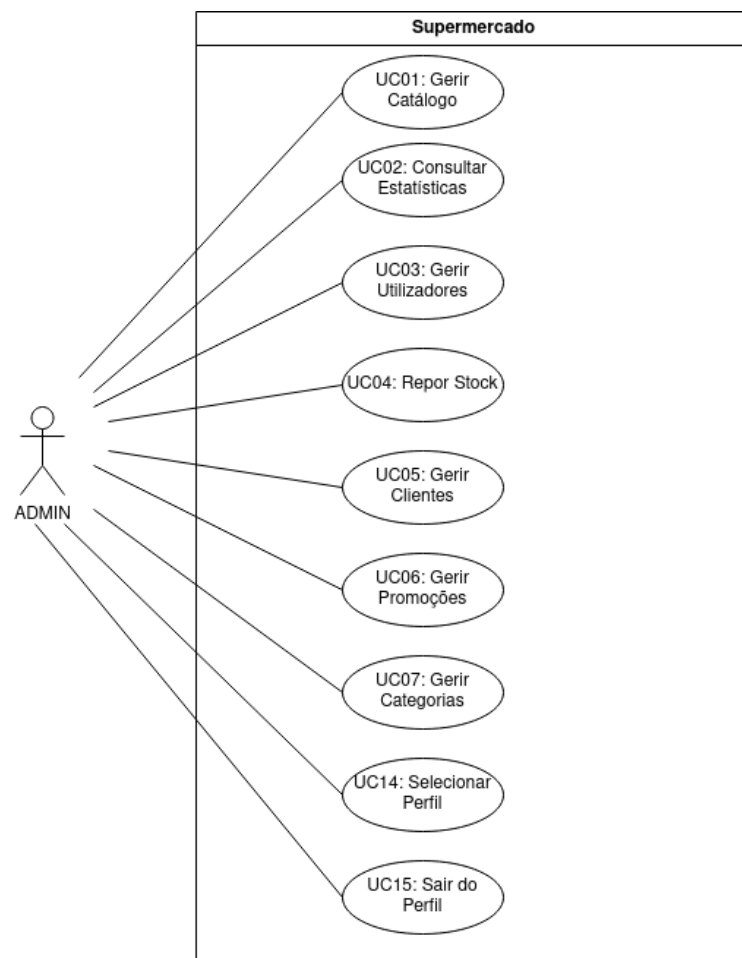


Fig. 11. Diagrama de Casos de Uso - Administrador

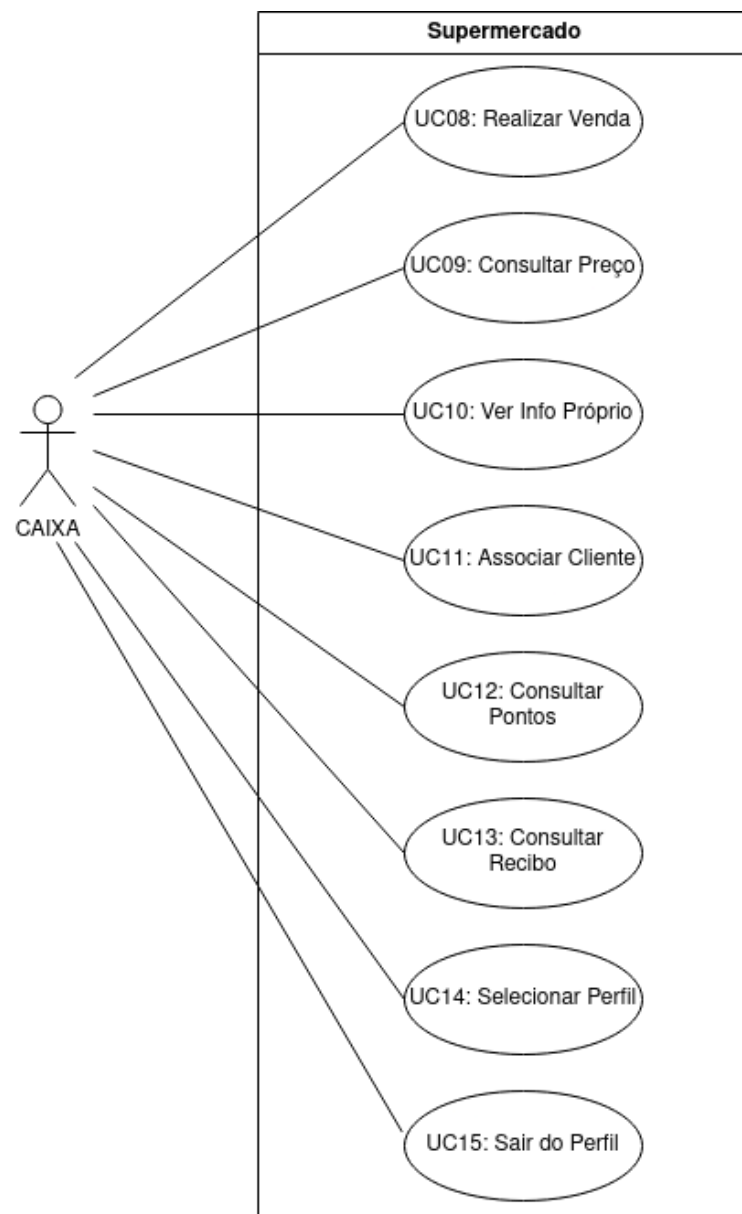


Fig. 12. Diagrama de Casos de Uso - Caixa

ID	Caso de Uso	Ator(es)	Descrição Resumida
UC01	Gerir Catálogo	ADMIN	Permite criar, editar, listar e remover PRODUTOS.
UC02	Consultar Estatísticas	ADMIN	Gera relatórios de faturação e volume por CAIXA ou global.
UC03	Gerir Utilizadores	ADMIN	Permite o registo e remoção de contas de perfil 'CAIXA'.
UC04	Repor Stock	ADMIN	Incrementa a quantidade disponível de um PRODUTO existente.
UC05	Gerir CLIENTES	ADMIN	Permite o registo, edição e remoção de perfis de CLIENTE.
UC06	Gerir PROMOÇÕES	ADMIN	Permite criar, editar e apagar PROMOÇÕES temporárias.
UC07	Gerir CATEGORIAS	ADMIN	Agrupar PRODUTOS para fins de IVA e Descontos.
UC08	Realizar VENDA	CAIXA	Inicia transação, adiciona itens, calcula total e regista pagamento.
UC09	Consultar Preço	CAIXA	Pesquisa e exhibe o preço unitário de um PRODUTO pelo seu ID.
UC10	Ver Info Própria	CAIXA	Exibe dados do funcionário e total de vendas faturado.
UC11	Associar CLIENTE	CAIXA	Durante a VENDA, permite ligar a transação a um CLIENTE.
UC12	Consultar Pontos	CAIXA	Visualizar quantidade de PONTOS do CLIENTE.
UC13	Consultar RECIBO	CAIXA	Consultar e visualizar o RECIBO de uma VENDA terminada.
UC14	Selecionar Perfil	ADMIN, CAIXA	Identifica o utilizador e o seu papel para aceder ao sistema.
UC15	Sair do Perfil	ADMIN, CAIXA	Termina a sessão do utilizador atual.

Table 1. Lista de Casos de Uso

5.1 Especificações de Casos de Uso

Actor	ADMIN
Use case name	Gerir Catálogo
Description	Permite criar, editar, listar e remover PRODUTOS.
Precondition	ADMIN selecionado no sistema.
Postcondition	Catálogo de PRODUTOS atualizado na base de dados.
Main flow	1. O ADMIN seleciona a opção "gestão do catálogo". 2. O sistema mostra as opções disponíveis (Criar, Editar, Listar, Remover). 3. O ADMIN seleciona "Criar". 4. O sistema pede os dados (nome, preço base, stock inicial, categoria). 5. O ADMIN introduz os dados. 6. O sistema valida os dados, cria um ID automático e grava. 7. O sistema confirma o sucesso da operação.
Alternative path	3.a. Listar: 1. O ADMIN seleciona "Listar". 2. O sistema apresenta a lista de todos os produtos e respetivos detalhes. 3. O UC termina. 3.b. Editar: 1. O ADMIN seleciona a opção "Editar". 2. O sistema pede o ID do produto. 3. O ADMIN escreve o ID. 4. O sistema mostra os dados atuais e pede novos dados. 5. O ADMIN insere os novos dados. 6. O sistema valida e atualiza o produto. 7. O UC termina. 3.c. Remover: 1. O ADMIN seleciona "Remover". 2. O sistema pede o ID do produto. 3. O ADMIN insere o ID. 4. O sistema confirmação. 5. O ADMIN confirma. 6. O sistema remove o produto 7. O UC termina.

UC01: Gerir Catálogo

Actor	ADMIN
Use case name	Consultar Estatísticas
Description	Permite visualizar relatórios de faturação global ou por CAIXA.
Precondition	ADMIN selecionado no sistema.
Postcondition	N/A
Main flow	1. O ADMIN seleciona a opção de "consulta de estatísticas". 2. O sistema pede o filtro (global ou por utilizador CAIXA). 3. O ADMIN escolhe o filtro. 4. O sistema cria e apresenta o relatório de faturação e quantidade de vendas.
Alternative path	N/A

UC02: Consultar Estatísticas

Actor	ADMIN
Use case name	Gerir Utilizadores
Description	Permite registar e remover contas de perfil 'CAIXA'.
Precondition	ADMIN selecionado no sistema.
Postcondition	Lista de CAIXAs é atualizada.
Main flow	1. O ADMIN seleciona a opção "gestão de utilizadores". 2. O sistema mostra as opções (Registar, Listar, Remover). 3. O ADMIN seleciona "Registar". 4. O ADMIN introduz os dados do novo CAIXA (nome). 5. O sistema verifica os dados e cria a conta de perfil 'CAIXA'. 6. O sistema confirma a criação com sucesso.
Alternative path	3.b. Remover: 1. O ADMIN seleciona a opção "Remover". 2. O sistema pede o ID do CAIXA. 3. O ADMIN escreve o ID. 4. O sistema pede confirmação. 5. O ADMIN confirma. 6. O sistema remove a conta e informa o sucesso. 7. O UC termina.

UC03: Gerir Utilizadores

Actor	ADMIN
Use case name	Repor Stock
Description	Aumenta a quantidade disponível de um PRODUTO existente.
Precondition	ADMIN selecionado no sistema.
Postcondition	Stock do PRODUTO é incrementado.
Main flow	1. O ADMIN pesquisa o PRODUTO pelo ID. 2. O sistema mostra os dados atuais do PRODUTO. 3. O ADMIN introduz a quantidade a adicionar ao stock. 4. O sistema atualiza o valor do stock na base de dados. 5. O sistema confirma o novo valor total de stock.
Alternative path	N/A

UC04: Repor Stock

Actor	ADMIN
Use case name	Gerir CLIENTES
Description	Permite registar, editar e remover perfis de CLIENTE.
Precondition	ADMIN selecionado no sistema.
Postcondition	Base de dados de CLIENTES atualizada.
Main flow	1. O ADMIN escolhe a opção "gestão de CLIENTES". 2. O sistema mostra as opções (Registar, Editar, Listar, Remover). 3. O ADMIN seleciona a opção "Registar". 4. O ADMIN introduz o NIF e o nome do CLIENTE. 5. O sistema valida o NIF, coloca o quantidade de PONTOS a zero e guarda o perfil. 6. O sistema confirma o sucesso da operação.
Alternative path	3.a. Listar: 1. O ADMIN seleciona a opção "Listar". 2. O sistema apresenta a lista de todos os CLIENTES (NIF, Nome, quantidade de PONTOS). 3. O UC termina. 3.b. Editar: 1. O ADMIN seleciona a opção "Editar". 2. O sistema solicita o NIF do CLIENTE. 3. O ADMIN insere o NIF. 4. O sistema mostra os dados atuais e solicita novo nome. 5. O ADMIN insere o novo nome. 6. O sistema atualiza o perfil. 7. O UC termina. 3.c. Remover: 1. O ADMIN seleciona a opção "Remover". 2. O sistema solicita o NIF do CLIENTE. 3. O ADMIN insere o NIF. 4. O sistema pede confirmação. 5. O ADMIN confirma. 6. O sistema remove o perfil e informa o sucesso. 7. O UC termina.

UC05: Gerir CLIENTES

Actor	ADMIN
Use case name	Gerir PROMOÇÕES
Description	Permite criar e configurar regras de descontos temporários.
Precondition	ADMIN selecionado no sistema.
Postcondition	Base de dados de PROMOÇÕES atualizada.
Main flow	1. O ADMIN escolhe a opção "Gestão de PROMOÇÕES". 2. O sistema mostra as opções (Criar, Listar, Remover). 3. O ADMIN seleciona "Criar". 4. O ADMIN define a percentagem de desconto e as datas de início e final. 5. O ADMIN associa a PROMOÇÃO a um PRODUTO ou CATEGORIA específica. 6. O sistema valida e guarda a regra de desconto. 7. O sistema confirma o sucesso.
Alternative path	3.a. Listar: 1. O ADMIN seleciona "Listar". 2. O sistema apresenta a lista de todas as PROMOÇÕES ativas e agendadas. 3. O UC termina. 3.b. Remover: 1. O ADMIN seleciona a opção "Remover". 2. O sistema solicita a identificação da promoção. 3. O ADMIN seleciona a promoção a remover. 4. O sistema pede confirmação. 5. O ADMIN confirma. 6. O sistema remove a regra de desconto e informa o sucesso. 7. O UC termina.

UC06: Gerir PROMOÇÕES

Actor	ADMIN
Use case name	Gerir CATEGORIAS
Description	Permite agrupar PRODUTOS em CATEGORIAS para find de impostos e descontos.
Precondition	ADMIN selecionado no sistema.
Postcondition	Lista de CATEGORIAS atualizada.
Main flow	1. O ADMIN escolhe a opção "gestão de categorias". 2. O sistema apresenta as opções (Criar, Listar, Remover). 3. O ADMIN seleciona "Criar". 4. O ADMIN introduz o nome da nova CATEGORIA e a taxa de IVA dessa CATEGORIA. 5. O sistema valida os dados e cria a CATEGORIA. 6. O sistema confirma a criação com sucesso.
Alternative path	3.a. Listar: 1. O ADMIN seleciona "Listar". 2. O sistema apresenta a lista de todas as categorias. 3. O UC termina. 3.b. Remover: 1. O ADMIN seleciona a opção "Remover". 2. O sistema solicita o ID da categoria. 3. O ADMIN insere o ID. 4. O sistema remove a categoria e informa o sucesso. 5. O UC termina.

UC07: Gerir CATEGORIAS

Actor	CAIXA
Use case name	Realizar VENDA
Description	Realiza uma VENDA de PRODUTOS a um CLIENTE ou a alguém que não é um CLIENTE.
Precondition	CAIXA selecionado no sistema.
Postcondition	VENDA registada, o stock atualizado e RECIBO emitido.
Main flow	1. O CAIXA seleciona a opção de iniciar uma nova VENDA. 2. O CAIXA introduz o ID e quantidade de cada PRODUTO. 3. O sistema valida o item, calcula o subtotal e apresenta-o. 4. O CAIXA termina a introdução de itens. 5. O sistema calcula o total final (aplica impostos e promoções). 6. O CAIXA regista o pagamento. 7. O sistema acaba a VENDA e emite o RECIBO.
Alternative path	6.a. Cancelar VENDA: 1. O CAIXA solicita o cancelamento da venda antes do pagamento. 2. O sistema pede confirmação. 3. O CAIXA confirma. 4. O sistema descarta os dados da venda em curso e liberta o stock reservado. 5. O UC termina.

UC08: Realizar VENDA

Actor	CAIXA
Use case name	Consultar Preço
Description	Pesquisa e mostra o preço de um PRODUTO pelo seu ID.
Precondition	CAIXA selecionado no sistema.
Postcondition	N/A (Consulta).
Main flow	1. O CAIXA escreve o ID do PRODUTO. 2. O sistema pesquisa o ID. 3. O sistema apresenta o nome e o preço do PRODUTO.
Alternative path	N/A

UC09: Consultar Preço

Actor	CAIXA
Use case name	Ver Info Própria
Description	Mostra os dados do utilizador CAIXA e o total de dinheiro das vendas acumulado.
Precondition	CAIXA selecionado no sistema.
Postcondition	N/A (Consulta).
Main flow	1. O CAIXA solicita a opção de ver a sua informação. 2. O sistema apresenta os dados do perfil (ID, nome) e o total faturado acumulado.
Alternative path	N/A

UC10: Ver Info Própria

Actor	CAIXA
Use case name	Associar CLIENTE
Description	Durante uma VENDA, associa o CLIENTE à transação.
Precondition	VENDA em curso.
Postcondition	CLIENTE está associado à VENDA.
Main flow	1. O CAIXA solicita a opção de "associação de um CLIENTE." 2. O CAIXA introduz o NIF do CLIENTE. 3. O sistema valida o NIF e mostra o nome correspondente. 4. O sistema liga o CLIENTE à VENDA atual.
Alternative path	N/A

UC11: Associar CLIENTE

Actor	CAIXA
Use case name	Consultar Pontos
Description	Permite consultar o quantidade de PONTOS atual de um CLIENTE.
Precondition	CAIXA selecionado no sistema.
Postcondition	N/A (Consulta).
Main flow	1. O CAIXA introduz o NIF do CLIENTE. 2. O sistema pesquisa o perfil correspondente. 3. O sistema apresenta o quantidade de PONTOS atual do CLIENTE.
Alternative path	N/A

UC12: Consultar Pontos

Actor	CAIXA
Use case name	Consultar RECIBO
Description	Permite visualizar uma venda já feita.
Precondition	CAIXA selecionado no sistema.
Postcondition	N/A (Consulta).
Main flow	1. O CAIXA seleciona uma das vendas que efetuou. 2. O sistema pesquisa os dados da venda. 3. O sistema cria e mostra o RECIBO detalhado no ecrã.
Alternative path	N/A

UC13: Consultar RECIBO

Actor	ADMIN, CAIXA
Use case name	Selecionar Perfil
Description	Permite ao utilizador do programa identificar-se (ADMIN ou CAIXA) e aceder ao menu de opções correspondente.
Precondition	Sistema no menu de seleção inicial.
Postcondition	Utilizador selecionado e menu de funções ativo.
Main flow	1. O utilizador seleciona o seu perfil (ADMIN ou um CAIXA específico) de uma lista apresentada. 2. O sistema verifica as permissões do perfil. 3. O sistema apresenta o menu de operações ao papel selecionado.
Alternative path	N/A

UC14: Selecionar Perfil

Actor	ADMIN, CAIXA
Use case name	Sair do Perfil
Description	Termina a sessão do utilizador atual.
Precondition	Utilizador com ADMIN selecionado OU CAIXA selecionado no sistema.
Postcondition	Sessão terminada e sistema regressa ao menu de seleção inicial.
Main flow	1. O utilizador seleciona a opção de "Logout". 2. O sistema limpa os dados de sessão atuais. 3. O sistema regressa ao menu de seleção de perfil inicial.
Alternative path	N/A

UC15: Sair do Perfil

5.2 Casos de Uso Implementados

Na terceira iteração, todos os 15 casos de uso foram implementados:

- **UC01** – Gerir Catálogo
- **UC02** – Consultar Estatísticas
- **UC03** – Gerir Utilizadores (CAIXAs)
- **UC04** – Repor Stock (integrado no UC01)
- **UC05** – Gerir CLIENTES
- **UC06** – Gerir PROMOÇÕES
- **UC07** – Gerir CATEGORIAS
- **UC08** – Realizar VENDA
- **UC09** – Consultar Preço
- **UC10** – Ver Info Própria (integrado no UC14)
- **UC11** – Associar CLIENTE (integrado no UC08)
- **UC12** – Consultar Pontos
- **UC13** – Consultar RECIBO (integrado no UC08)
- **UC14** – Selecionar Perfil
- **UC15** – Sair do Perfil

Nota: Os casos de uso UC04, UC10, UC11 e UC13 estão integrados noutros. O UC04 (Repor Stock) faz parte do menu de edição do UC01, o UC10 (Ver Info Própria) está integrado no menu do CAIXA (UC14), e os UC11 (Associar CLIENTE) e UC13 (Consultar RECIBO) estão integrados no fluxo da VENDA (UC08).

Os diagramas de sequência que se seguem mostram o fluxo interno (View → Controller → Service → Repository) para cada caso de uso.

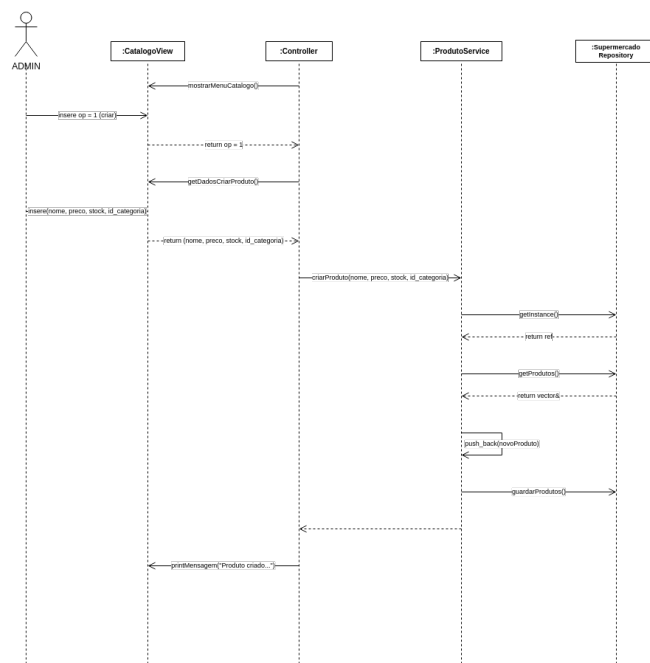


Fig. 13. UC01: Gerir Catálogo (Cenário de Criação)

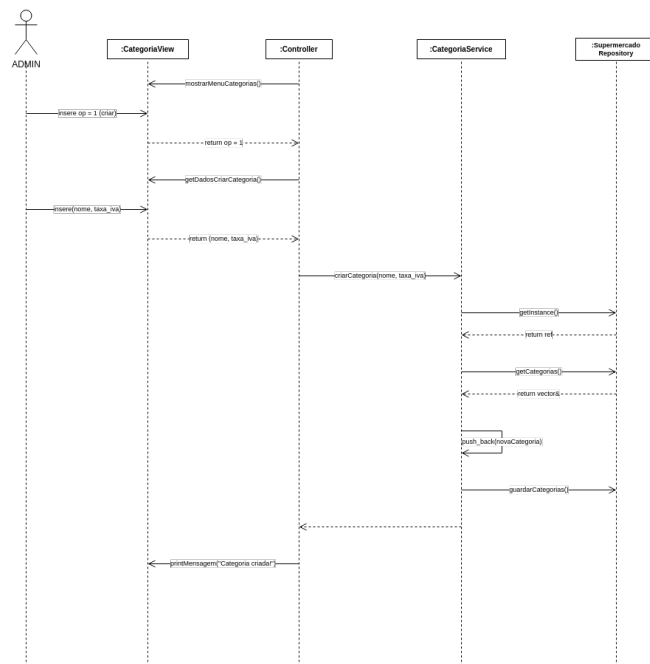


Fig. 14. UC07: Gerir CATEGORIAS

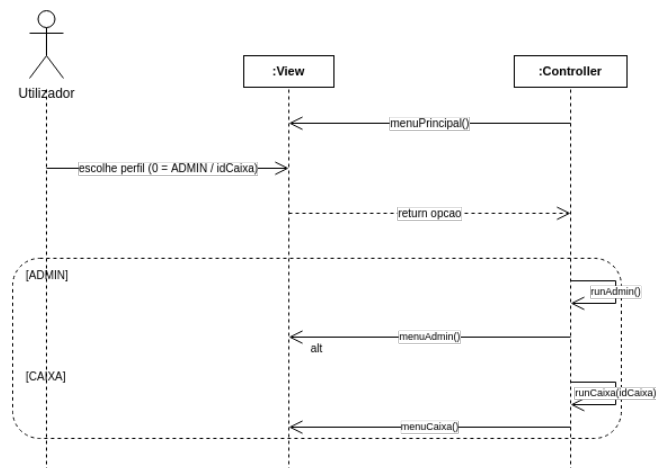


Fig. 15. UC14: Selecionar Perfil

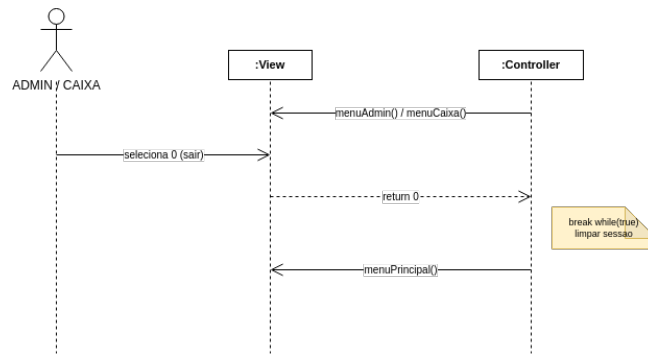


Fig. 16. UC15: Sair do Perfil

5.3 Restantes Diagramas de Sequência

Os diagramas seguintes foram criados na iteração anterior e os casos de uso correspondentes foram implementados na terceira iteração.

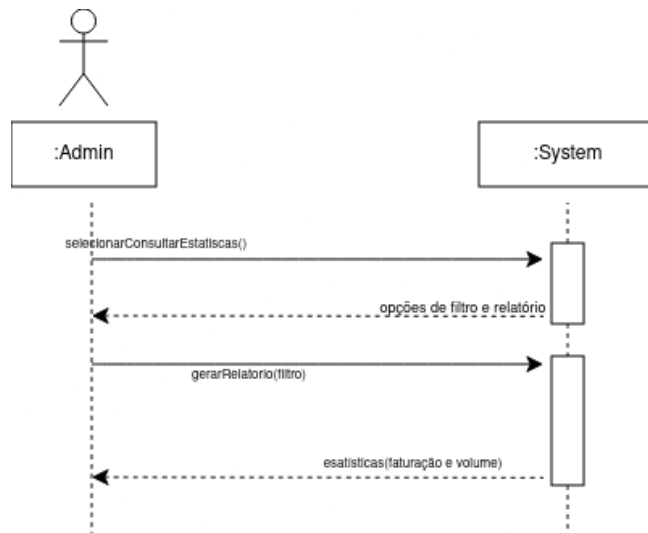


Fig. 17. UC02: Consultar Estatísticas

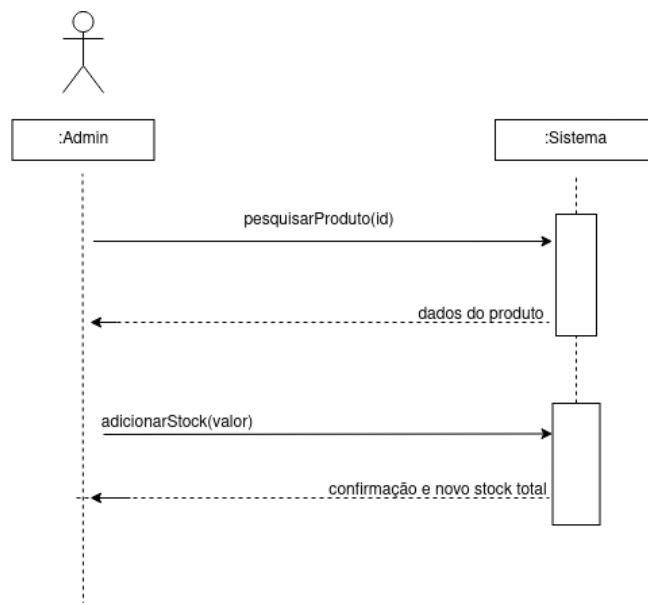


Fig. 18. UC04: Repor Stock

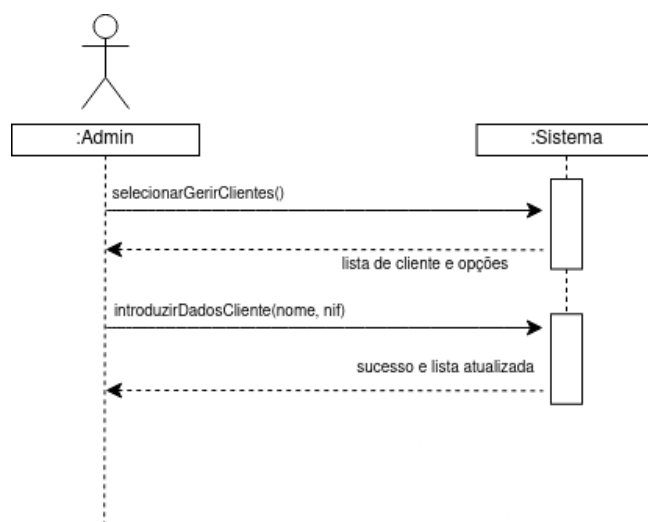


Fig. 19. UC05: Gerir CLIENTES (Cenário de Registo)

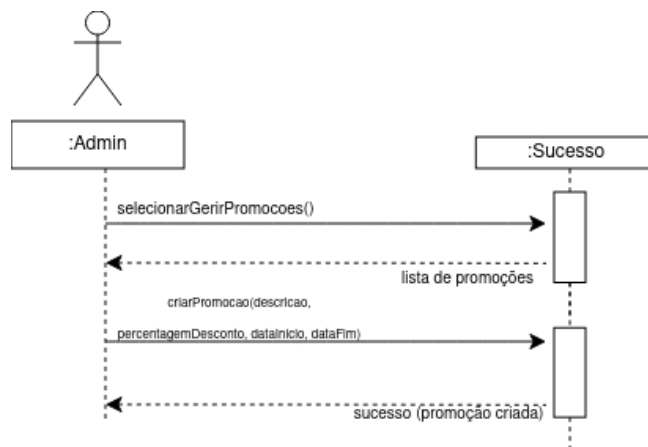


Fig. 20. UC06: Gerir PROMOÇÕES (Cenário de Criação)

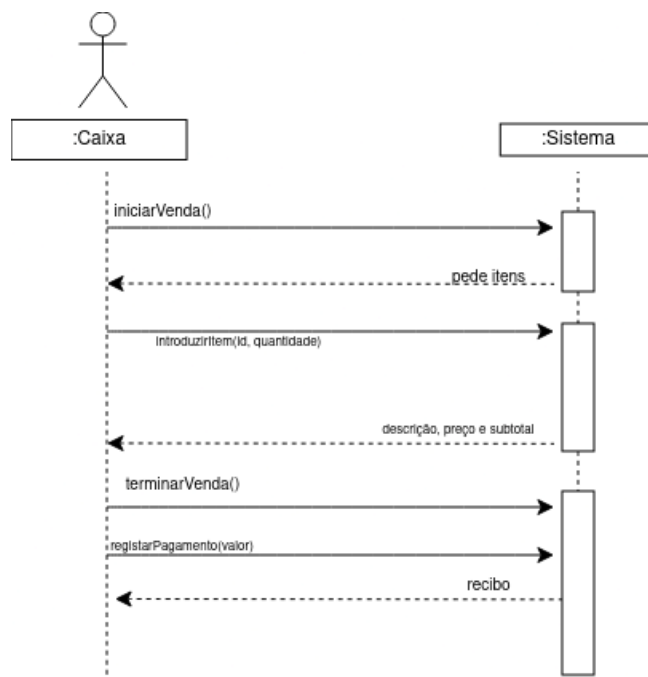


Fig. 21. UC08: Realizar VENDA

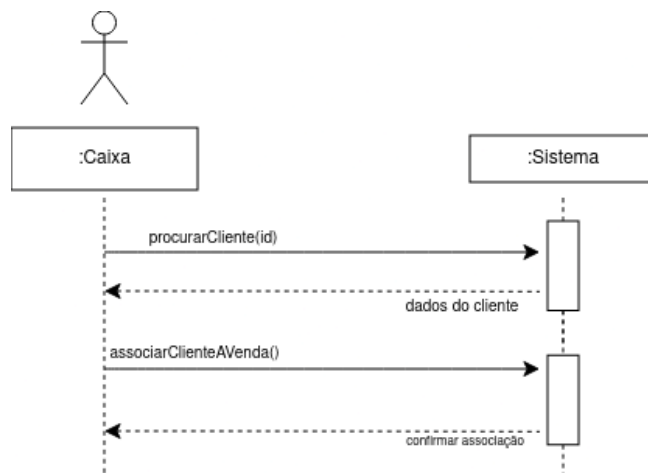


Fig. 22. UC11: Associar CLIENTE

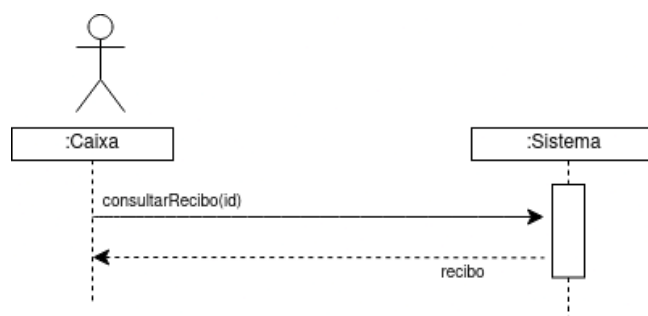


Fig. 23. UC13: Consultar RECIBO

6 Glossário

ADMIN Tipo de UTILIZADOR com o papel de gestão e controlo total das operações do sistema.

CAIXA Tipo de UTILIZADOR que atua como funcionário operador do sistema de vendas.

CATEGORIA Agrupamento de produtos com características semelhantes (ex: Bebidas, Talho, Peixaria).

CLIENTE Pessoa que compra PRODUTOS no supermercado, e que está registada no sistema para fidelização.

ESTATÍSTICA Relatório de desempenho de faturação e volume de vendas por período ou operador.

MÉTODO DE PAGAMENTO Forma como o cliente liquida o valor de uma venda (ex: Numerário, Cartão).

PONTO Unidade de medida de fidelização que ao ser acumulada por compras é possível converter para ter descontos.

PRODUTO Artigo com ID único, descrição, preço e stock.

PROMOÇÃO Desconto percentual aplicado a produtos ou categorias num certo intervalo de tempo.

RECIBO Comprovativo de uma venda concluída entregue ao cliente, que inclui o valor total pago, o método de pagamento e a lista de produtos comprados.

UTILIZADOR Entidade base do sistema que agrupa atributos comuns de identificação, da qual derivam os perfis específicos de operação.

VENDA Transação que associa produtos e um método de pagamento.

7 Testes Unitários

Os testes unitários foram implementados com o framework Google Test. O projeto de testes contém 40 testes organizados em 6 suites, uma por cada Service.

7.1 Estrutura dos Testes

Suite	Testes
ProdutoServiceTest	6
CategoriaServiceTest	6
ClienteServiceTest	5
CaixaServiceTest	6
VendaServiceTest	8
PromocaoServiceTest	9
Total	40

Table 2. Distribuição dos testes por Service

7.2 Cobertura de Testes

Cada suite cobre cenários de sucesso e de erro:

- **ProdutoService:** Criar produto válido, criar com nome vazio, criar com preço negativo, remover existente, remover inexistente, editar com -1.
- **CategoriaService:** Criar categoria válida, criar com nome vazio, criar com IVA negativo, remover existente, remover inexistente, editar.
- **ClienteService:** Criar cliente válido, criar com NIF duplicado, remover existente, remover inexistente, editar.
- **CaixaService:** Criar caixa válido, criar com nome vazio, remover existente, remover inexistente, editar, editar com -1.

- **VendaService:** Iniciar venda sem cliente, iniciar com cliente, iniciar com cliente inexistente, adicionar item sem venda ativa, adicionar com stock insuficiente, cancelar venda, consultar preço existente, consultar preço inexistente.
- **PromocaoService:** Criar promoção de produto, criar de categoria, percentagem inválida, data inválida, data fim antes de início, produto inexistente, remover existente, remover inexistente, editar.

7.3 Execução

```
cd Project/build
cmake .. && make supermercado_tester
./supermercado_tester
```

Resultado: 40/40 testes passam.

8 Conclusão

A segunda iteração foi dedicada ao design e à implementação do sistema. Usamos o padrão MVC estendido em mais camadas (Services, Exceptions, repo etc), o que nos permitiu organizar o código de forma clara e com responsabilidades bem definidas. No total, resultaram desta fase 10 classes de domínio (Model), 4 classes de interface (View), 2 Services, 1 Repository com padrão Singleton, 7 DTOs, 2 Mappers e 4 classes de exceção. Todo este design foi documentado através de 8 diagramas UML, um por cada aspeto relevante da arquitetura. Do ponto de vista da implementação, conseguimos concluir 4 dos 15 casos de uso identificados na iteração anterior: Gerir Catálogo (UC01), Gerir Categorias (UC07), Selecionar Perfil (UC14) e Sair do Perfil (UC15). Aproveitámos também para rever os diagramas de sequência, tornando-os mais detalhados e de acordo com o fluxo real do sistema, mostrando a informação que percorre as camadas, da View até ao Repository, passando pelo Controller e pelo Service. Para a próxima iteração, o objetivo é implementar os casos de uso que ainda faltam, com especial foco na gestão de Vendas, Clientes e Promoções. E realizar os testes unitários, utilizando o framework Google Test.

Bibliografia

1. ISEP: Fundamentos de Desenvolvimento de Software (FSOFT) - Material de Apoio (Aulas Teóricas e Guia do Projeto) (2025/2026)
2. ISO/IEC: ISO/IEC 14882:2017 - Programming languages – C++. <https://www.iso.org/standard/68564.html> (2017)